AWS SERIES · MODULE 1 OF 8

AWS Networking: The Parts You Actually Need

A beginner's guide: what an IP address is, why more devices need more IPs, how AWS is laid out, how a company gives each team its own network, and how traffic moves — with only the depth a GenAI engineer needs.

PA

Tutor: Prudhvi Akella

Lead Data & AI Engineer · 13+ years in data engineering and AI architecture

Services: Regions · Availability Zones · AWS Organizations · Amazon VPC · Subnets · Route Tables · Internet / NAT Gateway · Security Groups · VPC Endpoints · Transit Gateway

Sample company used throughout: **AcmeCorp** — teams HR, Finance and IT, AWS in Mumbai (`ap-south-1`)

1. What an IP address is
2. Every device needs an IP
3. What takes an IP in AWS
4. Groups of IPs: CIDR ranges
5. Sizing a range from device count
6. AWS around the world: Regions and AZs
7. Accounts: one per team
8. VPC: your private network
9. IP plan for the whole company
10. Subnets: public and private
11. Route tables: the signposts
12. Internet Gateway: the door to the internet
13. NAT Gateway: private devices going out
14. Internet Gateway vs NAT Gateway
15. Security groups: the firewall
16. Reaching AWS services privately
17. Joining team networks
18. Flow 1: user → HR portal
19. Flow 2: server → internet
20. Flow 3: GenAI app → Bedrock and S3
21. Flow 4: HR app → IT service
22. What a GenAI engineer must know
23. What you can skip for now
24. Troubleshooting in 4 checks
25. Build it yourself: what you will build
26. Before you start: create an AWS account (console)
27. Before you start: set up the command line
28. Build 1 · Create the VPC
29. Build 2 · Create four subnets
30. Build 3 · Create and attach the Internet Gateway
31. Build 4 · Public route table: 0.0.0.0/0 → IGW
32. Build 5 · NAT Gateway and private route table
33. Build 6 · S3 gateway endpoint
34. Build 7 · Security groups
35. Build 8 · Launch two servers
36. Build 9 · Check that it works
37. Build 10 · Clean up (same day)
38. Key takeaways

1 What an IP address is

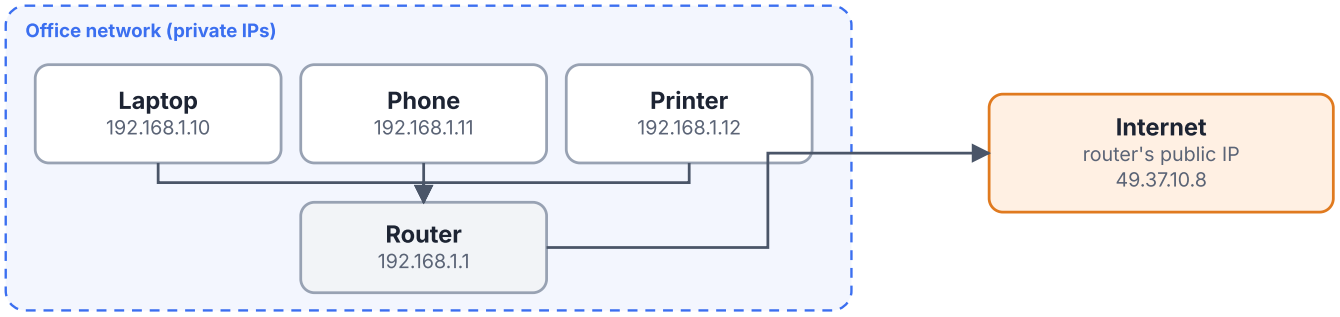


Figure 1. Each device on a network has its own IP address. Data is sent from one IP to another.

An **IP address** is a number that identifies a device on a network, for example `192.168.1.10`. When one device sends data to another, the data carries two IPs: **from** (source) and **to** (destination). Network equipment reads the destination IP and forwards the data towards it.

Part	Meaning
Format (IPv4)	Four numbers from 0 to 255, separated by dots: `10.2.16.25`
Unique	Two devices on the same network cannot have the same IP
Private IP	Used inside a network (home, office, AWS VPC). Not reachable from the internet. Ranges: `10.x.x.x`, `172.16–31.x.x`, `192.168.x.x`
Public IP	Reachable from the internet. Unique across the whole world

> Your laptop has a private IP inside the office. When it goes to the internet, the router swaps it for the router's single public IP. That is why one public IP can serve many devices.

2 Every device needs an IP



Figure 2. More devices → more IPs. The range must be big enough for all of them, plus growth.

The rule is simple: **every device that sends or receives data needs its own IP address.** To be exact, every **network interface** needs one. Most devices have one network interface, so: one device, one IP.

1. **1 server** = 1 IP.
2. **10 servers** = 10 IPs.
3. **A server with 2 network interfaces** = 2 IPs (rare; you will not need this early on).
4. **If the range runs out of free IPs, no new device can start.** In AWS you see errors like `InsufficientFreeAddressesInSubnet`, and scaling stops.

This is the most important idea in this module: **the number of IPs you plan for limits how many things you can run.** Too few IPs is a real outage risk when your app scales.

3 What takes an IP in AWS

In AWS a "device" is not only a server. Many services quietly take IPs from your network:

AWS thing (inside your VPC)	IPs it uses
EC2 server	1 per server (more if you add network interfaces)
ECS task (container) on Fargate	1 per running task
EKS pod (Kubernetes)	1 per pod by default — this adds up fast
Lambda function attached to a VPC	A few shared IPs per subnet + security group, not one per call
Load balancer (ALB)	At least 1 per AZ; more as traffic grows
RDS database	1 per database instance (2 with a standby)
VPC interface endpoint (e.g. Bedrock)	1 per AZ it is placed in
NAT Gateway	1 or more

Example: count the IPs for the HR app in one AZ

Item	Count	IPs
App containers (ECS tasks) at peak	200	200
Load balancer nodes	up to 8	8
Database + standby	2	2
Endpoints (Bedrock, Secrets Manager)	2	2
Lambda in VPC	—	~5
Total		**~217**

Containers and Kubernetes pods use the most IPs. If your GenAI app runs on ECS or EKS, size the subnet for the **peak** number of tasks or pods, not the normal number.

4 Groups of IPs: CIDR ranges

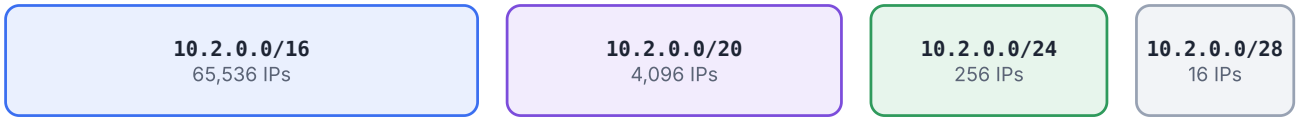


Figure 3. The number after the slash says how big the range is. Smaller number = bigger range.

We never hand out IPs one by one. We reserve a **range** and write it as a start address plus a slash number. This is called **CIDR**. You do not need the bit maths — just remember this table:

CIDR	IPs in it	Usable in an AWS subnet	Example range
`10.2.0.0/16`	65,536	—	`10.2.0.0` to `10.2.255.255`
`10.2.16.0/20`	4,096	4,091	`10.2.16.0` to `10.2.31.255`
`10.2.0.0/24`	256	251	`10.2.0.0` to `10.2.0.255`
`10.2.0.0/28`	16	11	`10.2.0.0` to `10.2.0.15`

- Each step **down** by 1 in the slash number **doubles** the range (/24 = 256, /23 = 512, /22 = 1,024).
- AWS keeps **5 IPs** in every subnet for itself, so a /24 gives you 251.
- In AWS, a VPC can be from /16 (biggest) to /28 (smallest).

5 Sizing a range from device count

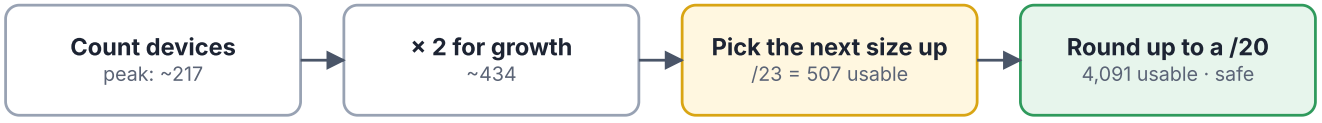


Figure 4. From device count to subnet size.

1. **Count the devices** at the busiest time (section 3).
2. **Multiply by 2 to 4** for growth and for rolling deployments (old and new containers run together for a while).
3. **Pick a CIDR that is bigger** using the table in section 4.
4. **When unsure, go bigger.** Private IPs are free. Changing a subnet later means rebuilding it.

> Simple defaults that rarely go wrong: **VPC = /16**, **app subnets = /20**, **load balancer and database subnets = /24**.

6 AWS around the world: Regions and AZs

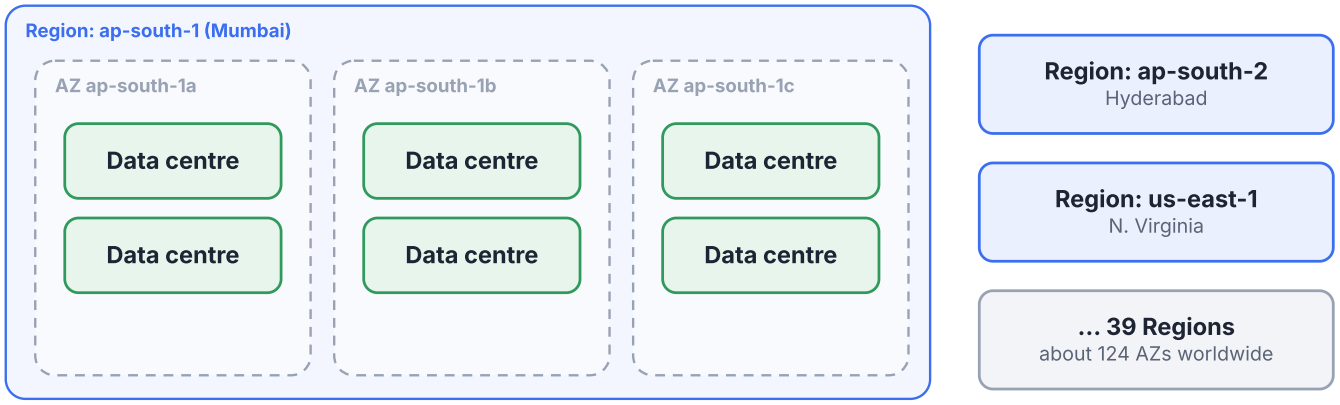


Figure 5. A Region is a group of AZs. Each AZ is one or more data centres with its own power and network.

Word	What it is	Why you care
Region	A city/area where AWS has data centres, e.g. Mumbai `ap-south-1`	Your data stays in the Region you choose. Pick one near your users and allowed by data rules
Availability Zone (AZ)	One or more data centres inside a Region, kilometres apart from other AZs	If one AZ fails, the others keep running. Use at least 2
Data centre	The actual building	You never choose it — AWS does

> Not every AI model is in every Region. Before choosing a Region for a GenAI app, check that the Bedrock models you need are available there.

> Counts change often. Check **aws.amazon.com/about-aws/global-infrastructure** for the latest.

7 Accounts: one per team

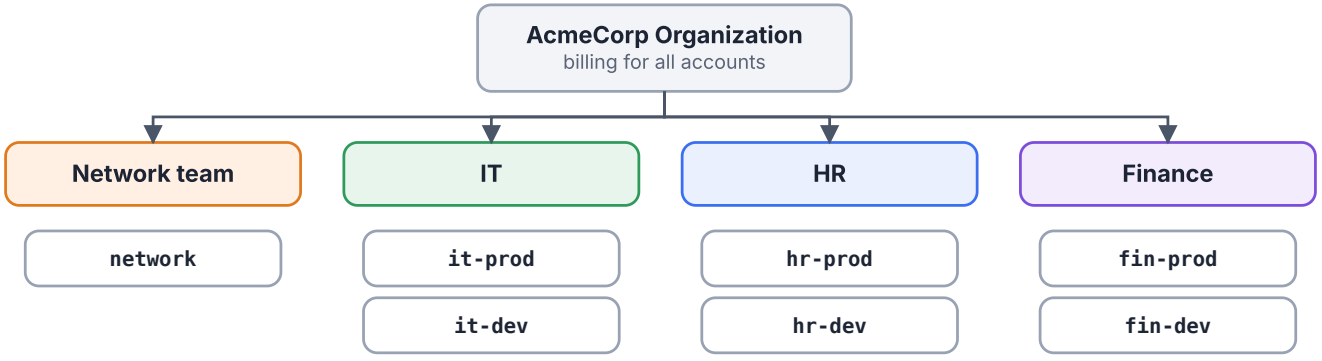


Figure 6. AcmeCorp's AWS Organization: one account per team per environment.

An **AWS account** is a sealed box. People, servers and networks in one account cannot see another account unless you connect them on purpose. Companies use this to keep teams apart.

Why one account per team per environment	Example
Mistakes stay small	A bad change in `hr-dev` cannot break `hr-prod`
Clear bills	You can see exactly what HR spends
Clear access	Finance staff get access only to Finance accounts

AWS Organizations puts all these accounts under one company umbrella. The **Network team** owns the shared connections; each team owns its own network inside its account.

8 VPC: your private network

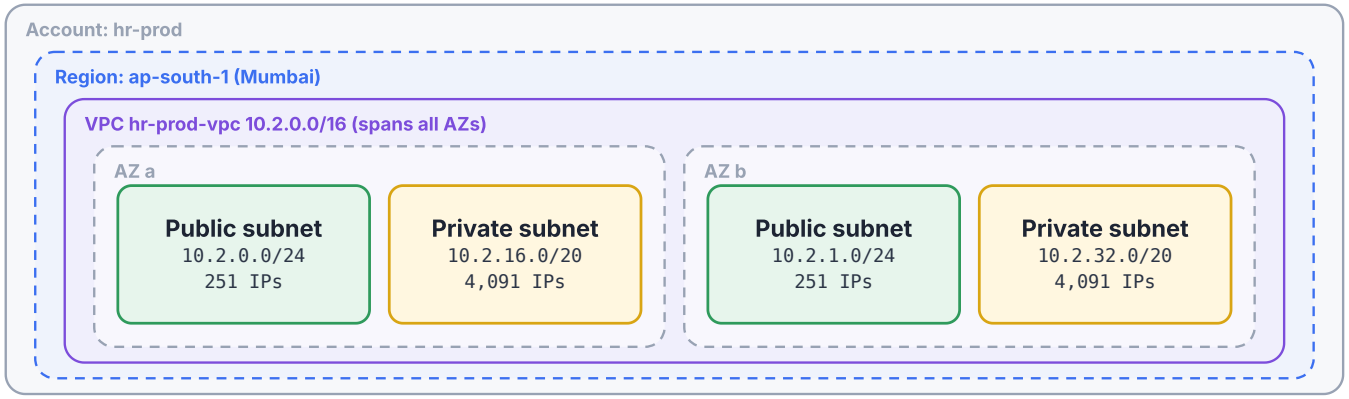


Figure 7. Account → Region → VPC → AZ → subnet. Each sits inside the one around it.

A **VPC (Virtual Private Cloud)** is your own private network inside one account and one Region. You give it an IP range (e.g. `10.2.0.0/16`). Every device you launch in it gets an IP from that range.

Thing	Lives in	Example
VPC	One account, one Region, all AZs	`hr-prod-vpc` = `10.2.0.0/16`
Subnet	One AZ, inside the VPC	`10.2.16.0/20` in AZ a
Device	One subnet — gets an IP from it	App container `10.2.16.25`

9 IP plan for the whole company

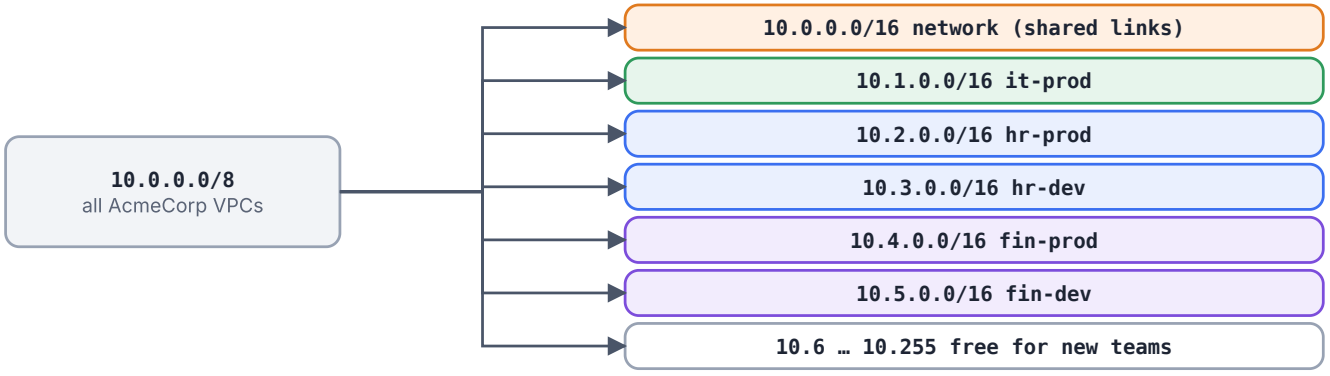


Figure 8. AcmeCorp's plan: one /16 per VPC, all from 10.0.0.0/8. No two ranges overlap.

If two VPCs use the same range, they can **never** be connected — a packet to `10.2.16.25` would not know which VPC to go to. So the company plans all ranges **once**, before any team builds.

1. **Reserve one big block for all of AWS:** `10.0.0.0/8`.
2. **Give each team VPC its own /16:** HR-prod `10.2.0.0/16`, Finance-prod `10.4.0.0/16`, and so on.
3. **Keep the office on a different block,** e.g. `172.16.0.0/16`, so it can connect later.
4. **Write it down in one place.** AWS **VPC IPAM** can store the plan and hand out the next free range.

10 Subnets: public and private

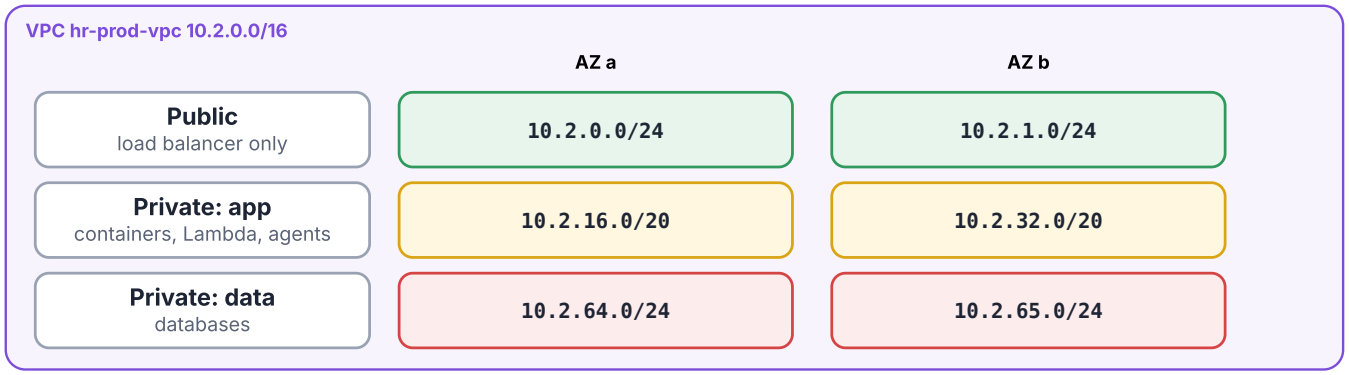


Figure 9. HR-prod's subnets: three tiers, repeated in two AZs. Sizes follow the device count.

Subnet type	What makes it that type	Put here
Public	Its route table has a path to the Internet Gateway	Only load balancers
Private	No direct path from the internet	Servers, containers, Lambda, AI agents, databases

- The app subnets are **/20** (4,091 IPs) because containers come and go and need many IPs (sections 3 and 5).
- Public and database subnets are **/24** (251 IPs) because they hold few devices.
- Each tier is repeated in a second AZ so the app survives if one AZ fails.
- Everything from `10.2.66.0` upwards is left free for later.

> Rule of thumb: **only the load balancer is public.** Your app, your AI agent and your database are always private.

11 Route tables: the signposts

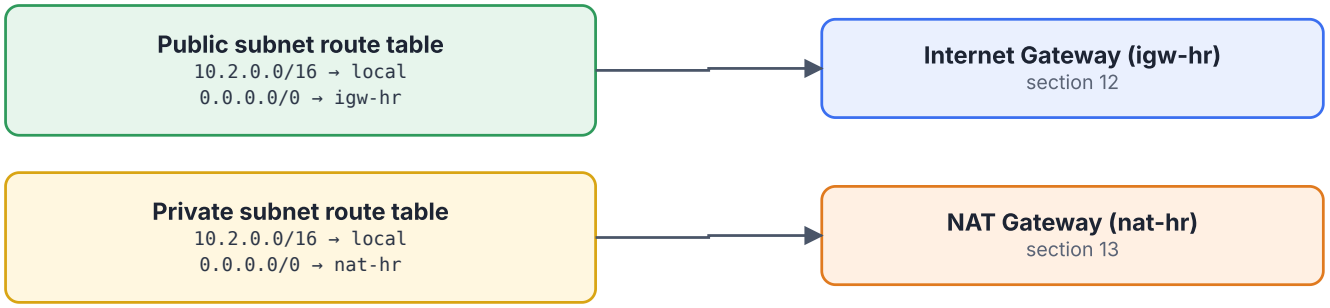


Figure 10. Every subnet has a route table. It decides the next stop for each packet.

A **route table** is a short list: "for this destination IP range, send the packet here". Each subnet uses one route table. The most specific match wins.

Route	Meaning
`10.2.0.0/16 → local`	Destination is inside the VPC: deliver directly. Every route table has this
`0.0.0.0/0 → igw-hr`	Everything else: send to the Internet Gateway. **This line makes a subnet public**
`0.0.0.0/0 → nat-hr`	Everything else: send to the NAT Gateway. Used in private subnets

1. **Packet to `10.2.64.10`** → inside `10.2.0.0/16` → `local` → delivered inside the VPC.
2. **Packet to `142.250.1.1` (internet)** → only `0.0.0.0/0` matches → IGW (public subnet) or NAT (private subnet).

12 Internet Gateway: the door to the internet

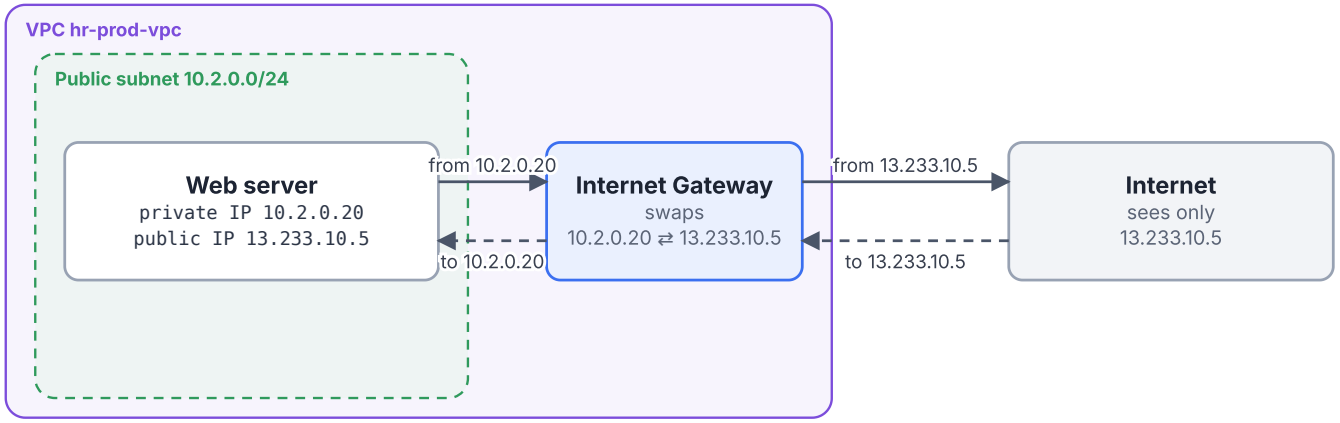


Figure 11. The server only knows its private IP. The Internet Gateway swaps it for the public IP on the way out, and back on the way in.

An **Internet Gateway (IGW)** connects a VPC to the internet. Traffic can go **out and come in** through it. You attach **one IGW per VPC**. AWS runs it for you: no size to choose, no servers, no extra charge for the IGW itself.

What it does to the IP address

A device in AWS never sees its own public IP. Its network card has only the private IP (`10.2.0.20`). The IGW keeps a one-to-one link between the private IP and the public IP (`13.233.10.5`) and swaps them:

Direction	Before the IGW	After the IGW
Going out	from `10.2.0.20` → to `142.250.1.1`	from `13.233.10.5` → to `142.250.1.1`
Coming in	from `142.250.1.1` → to `13.233.10.5`	from `142.250.1.1` → to `10.2.0.20`

A device is reachable from the internet only if all 3 are true

#	Requirement	Where you set it
1	The VPC has an IGW attached, and the subnet's route table has `0.0.0.0/0 → igw`	VPC → Route tables
2	The device has a **public IP** (auto-assigned or an Elastic IP)	Subnet setting / EC2 launch
3	The device's **security group** allows the port	Security groups

Missing any one of the three = no internet. This is the first thing to check when "my server is not reachable".

In good designs, the only things with a public IP are **load balancers**. Your app servers and AI agents sit in private subnets and never get a public IP. AWS also charges for every public IPv4 address, so fewer is cheaper.

13 NAT Gateway: private devices going out

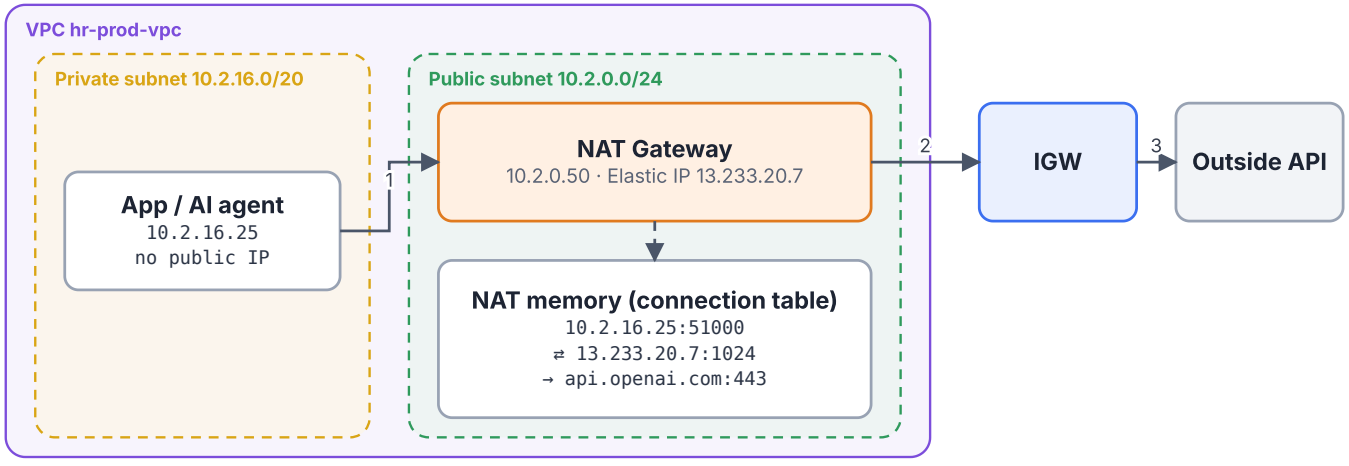


Figure 12. A private app goes out through the NAT Gateway. The NAT remembers who asked, so only replies can come back.

Private devices still need the internet: to download packages (`pip install`), call outside APIs, or reach a model provider that is not on AWS. They have no public IP, so the IGW alone cannot help them. A **NAT Gateway** (NAT = Network Address Translation) solves this.

How it works, step by step

Hop	Source IP	Destination IP	What happens
1 · app → NAT	`10.2.16.25`	`api.openai.com`	Private route table: `0.0.0.0/0 → nat-hr`
2 · NAT → IGW	`13.233.20.7`	`api.openai.com`	NAT swaps the source to its Elastic IP and **writes it in its memory**
3 · out to internet	`13.233.20.7`	`api.openai.com`	The API sees only the NAT's IP
4 · reply → NAT	`api.openai.com`	`13.233.20.7`	NAT finds the entry in its memory
5 · NAT → app	`api.openai.com`	`10.2.16.25`	Swapped back and delivered to the app

> **Why nothing can come in:** a new connection from the internet to `13.233.20.7` has no entry in the NAT's memory, so the NAT drops it. NAT = **out and replies only**.

Two kinds of NAT Gateway

	Zonal NAT Gateway (classic)	Regional NAT Gateway (new, late 2025)
Lives in	One public subnet in one AZ	The whole VPC; spreads across AZs by itself
Needs a public subnet?	Yes	No
For high availability	Create one per AZ, and point each AZ's private route table at its own NAT	One NAT is enough
Route table entry	`0.0.0.0/0 → nat-in-same-AZ`	`0.0.0.0/0 → nat-hr` everywhere

Cost — the part that surprises people

- You pay **per hour** for each NAT Gateway, **plus per GB** of data it processes.
- A GenAI app that downloads models or sends large files through NAT can run up a big bill.
- Traffic to AWS services (S3, Bedrock, DynamoDB …) does not need NAT. Use **VPC endpoints** instead (section 16).
- Delete lab NAT Gateways when you finish — they bill even when idle.

14 Internet Gateway vs NAT Gateway

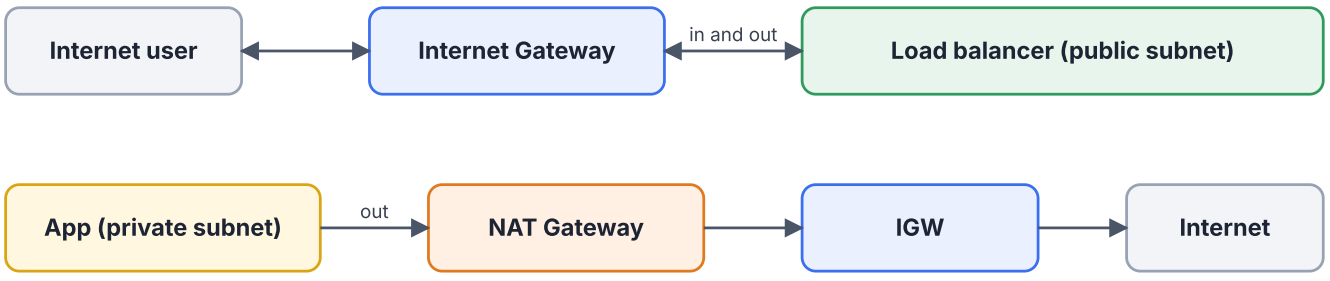


Figure 13. Side by side: IGW lets traffic in and out; NAT lets private devices out only.

	Internet Gateway	NAT Gateway
Purpose	Connect the VPC to the internet	Let **private** devices reach the internet
Direction	In and out	Out, plus replies only
Used by	Public subnets (load balancers)	Private subnets (apps, agents, Lambda)
Device needs a public IP?	Yes	No — the NAT's Elastic IP is used
How many	One per VPC	One per AZ (zonal) or one per VPC (regional)
Cost	No charge for the gateway itself	Per hour + per GB
Works alone?	Yes	No — the NAT sends traffic out through the IGW

> Remember: **IGW = in and out, for devices with a public IP. NAT = out only, for devices with a private IP — and the NAT itself goes out through the IGW.**

15 Security groups: the firewall



Figure 11. Each layer only accepts traffic from the layer in front of it.

A **security group** is a firewall around each device. It lists what traffic is **allowed in**. Anything not listed is blocked. Replies to allowed traffic are let back automatically.

Security group	Allow in	Meaning
`alb-sg`	Port 443 from anywhere (`0.0.0.0/0`)	Anyone can open the website over HTTPS
`app-sg`	Port 8080 from `alb-sg`	Only the load balancer can reach the app
`db-sg`	Port 5432 from `app-sg`	Only the app can reach the database

> Use a **security group as the source** (e.g. "from `app-sg` ") instead of IPs. Containers get new IPs all the time; the group name stays the same.

16 Reaching AWS services privately

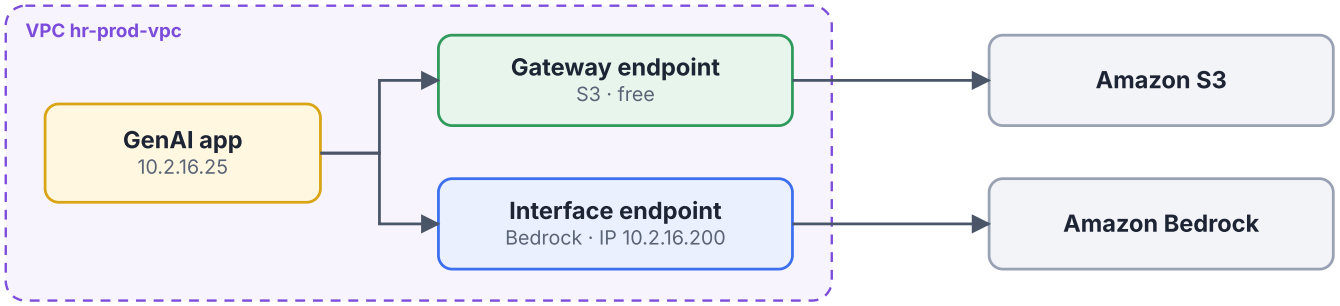


Figure 12. Endpoints give a private path from your VPC to AWS services. No NAT, no internet.

AWS services like S3, Bedrock, SQS and Secrets Manager live **outside** your VPC. A private app reaches them either through the NAT Gateway (costs money, goes via the internet path) or through a **VPC endpoint** (private).

Endpoint type	Works for	How	Cost
Gateway endpoint	S3, DynamoDB	Adds a line to your route table	Free
Interface endpoint	Bedrock, SQS, SNS, Secrets Manager and most others	Puts a network interface **with an IP from your subnet** in each AZ	Per hour + per GB

> An interface endpoint is another "device" in your subnet — it takes an IP (section 3). This is why endpoints count when you size subnets.

17 Joining team networks

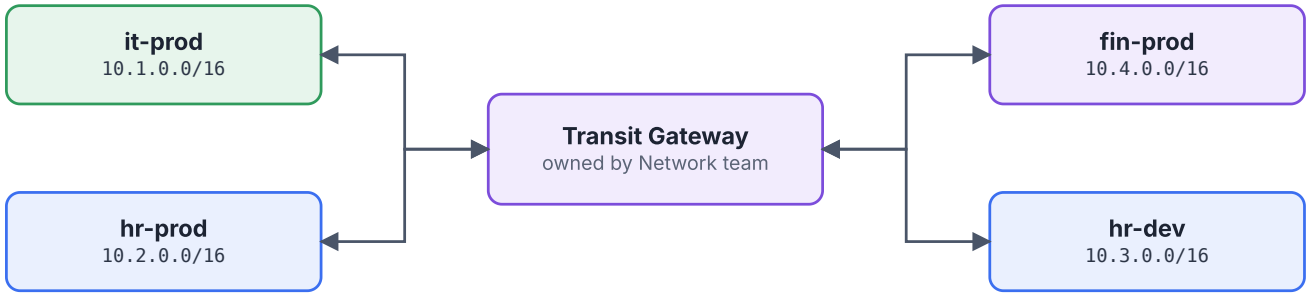


Figure 13. A Transit Gateway is a central router. Team VPCs connect to it, not to each other.

Each team's VPC is separate. When HR's app needs an IT service, the VPCs must be joined. Large companies use one **Transit Gateway** as a hub: every VPC connects to it once.

- HR adds one route: `10.0.0.0/8 → Transit Gateway` ("other company networks: go to the hub").
- The Network team decides which VPCs may talk. For example, **dev VPCs cannot reach prod VPCs**.
- This only works because every VPC has its own non-overlapping range (section 9).
- For just two VPCs, a simpler **VPC peering** link is enough.

18 Flow 1: user → HR portal

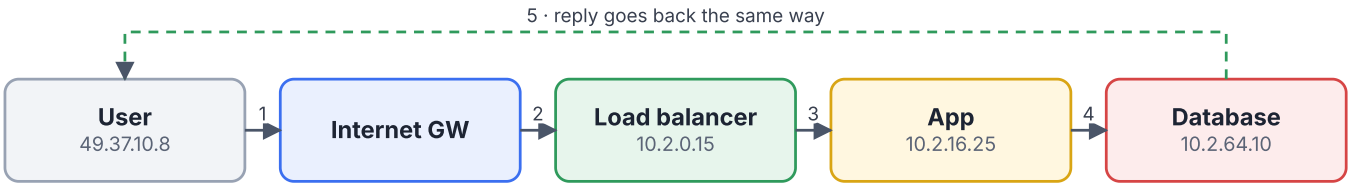


Figure 14. Flow 1 — a user opens the HR portal. Numbers match the steps.

1. **The user's browser sends a request** from `49.37.10.8` to the load balancer's public address (found via DNS).
2. **The Internet Gateway lets it in** because the public subnet's route table points to it.
3. **The load balancer (`10.2.0.15`) forwards it** to the app at `10.2.16.25:8080` . `app-sg` allows the load balancer.
4. **The app queries the database** at `10.2.64.10:5432` . `db-sg` allows the app.
5. **The answer travels back** database → app → load balancer → user.

19 Flow 2: server → internet

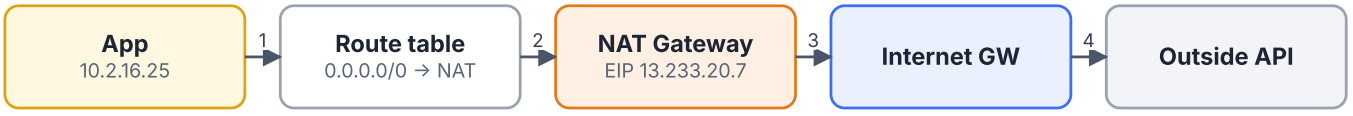


Figure 15. Flow 2 — a private server calls an outside API.

1. **The app sends a request to an outside API.** The destination is not inside `10.2.0.0/16` , so `0.0.0.0/0 → NAT` is used.
2. **The NAT Gateway swaps the source IP** `10.2.16.25` for its own public IP and remembers it.
3. **The request goes out through the Internet Gateway.** The API only ever sees the NAT's public IP.
4. **The reply comes back to the NAT,** which passes it to `10.2.16.25` . Nobody outside can start a new connection in.

20 Flow 3: GenAI app → Bedrock and S3

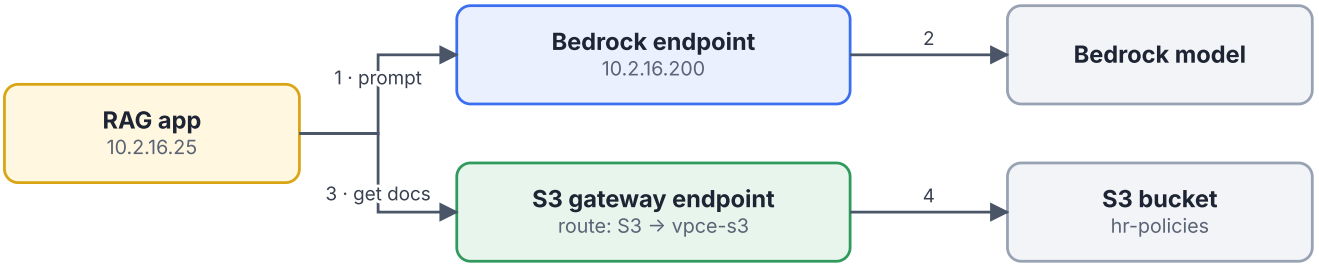


Figure 16. Flow 3 — a GenAI app calls Bedrock and reads documents from S3, both privately.

1. **The app calls Bedrock.** Private DNS makes the Bedrock address resolve to the endpoint's IP `10.2.16.200` inside the VPC.
2. **The endpoint passes the call to Bedrock** over the AWS network. No NAT, no internet.
3. **The app reads documents from S3.** The route table sends S3 traffic to the gateway endpoint.
4. **S3 returns the files** the same private way.

> Common beginner error: a Lambda or agent in a private subnet **times out** calling Bedrock or S3. Cause: no NAT and no endpoint, so there is no path out. Fix: add the endpoint (or a NAT).

21 Flow 4: HR app → IT service

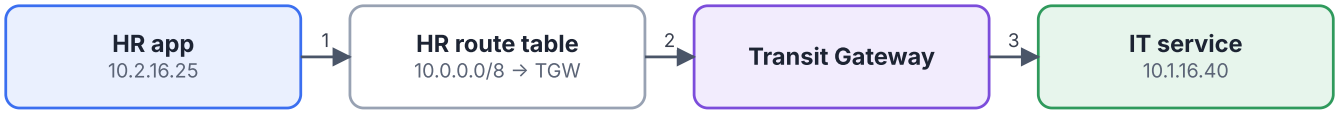


Figure 17. Flow 4 — the HR app calls an IT service in another team's VPC.

1. **The HR app sends to `10.1.16.40`.** That IP is not in HR's `10.2.0.0/16`, but matches `10.0.0.0/8 → Transit Gateway`.
2. **The Transit Gateway knows `10.1.0.0/16` belongs to it-prod** and forwards the packet there.
3. **The IT service's security group allows HR's range** (`10.2.0.0/16`) on its port. The reply returns the same way.

22 What a GenAI engineer must know

You need to …	Because …
Put apps, agents and Lambdas in **private** subnets	Nothing should be reachable from the internet except the load balancer
Give a private app a way out: **endpoints** (Bedrock, S3) and/or **NAT**	Otherwise calls to models and APIs time out
Size subnets for **peak** containers / pods	Running out of IPs stops scaling

You need to …	Because …
Open only the needed ports with **security groups**	It is your main firewall
Know your VPC's range and ask the Network team before choosing one	Overlapping ranges can never be connected

23 What you can skip for now

These matter to network specialists. Know that they exist; you can learn them when a project needs them.

Topic	One line
Network ACLs	A second, subnet-level firewall. Security groups are enough for most apps
Transit Gateway route tables	How the Network team separates dev from prod in detail
VPN and Direct Connect	How the office network is linked to AWS
Shared VPC	One VPC whose subnets are shared to many accounts
Route 53 Resolver endpoints	Making internal names work between office and AWS
IPv6	The newer, much larger address format

24 Troubleshooting in 4 checks

When something cannot connect (usually you see a **timeout**), check in this order:

#	Ask	Fix
1	Does the **security group** on the target allow my port and source?	Add an inbound rule
2	Does my subnet's **route table** have a path to the target?	Add `→ NAT`, `→ endpoint`, or `→ Transit Gateway`
3	For internet access: is there a **NAT** (private) or **Internet Gateway** (public)?	Add the missing gateway
4	Did the subnet **run out of IPs**?	Check "Available IPv4 addresses" on the subnet; use a bigger subnet

> **VPC Reachability Analyzer** in the console tests a path for you and tells you which step blocks it.

25 Build it yourself: what you will build

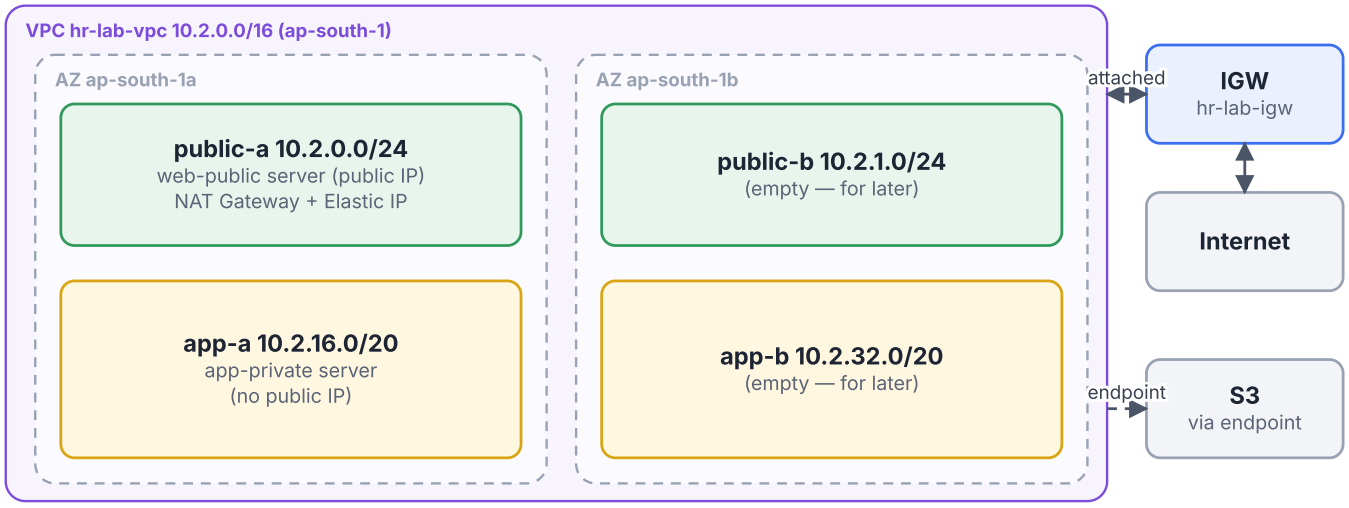


Figure 18. What you will build: a small copy of HR-prod in one Region, two AZs.

These steps build the network from this document, piece by piece. Every step is shown twice: **in the AWS Console** (clicking) and **on the command line** (AWS CLI), with separate commands for Mac and Windows.

#	You create	Proves
1	VPC `hr-lab-vpc` = `10.2.0.0/16`	Your private network (section 8)
2	4 subnets: public-a, public-b, app-a, app-b	Ranges and sizes (sections 4, 10)
3	Internet Gateway, attached	The door to the internet (section 12)
4	Public route table `0.0.0.0/0 → IGW`	What makes a subnet public (section 11)
5	NAT Gateway + private route table	Private devices going out (section 13)
6	S3 gateway endpoint	Reaching AWS privately (section 16)
7	Security groups `web-sg`, `app-sg`	The firewall (section 15)
8	Two servers: `web-public`, `app-private`	One device = one IP (section 2)
9	Checks	IGW in, NAT out, IPs used
10	Clean up	No surprise bill

Cost: the NAT Gateway, the public IPs and the servers are billed per hour. Finishing the lab and cleaning up in one sitting (about an hour) costs very little. **Do step 10 the same day.**

26 Before you start: create an AWS account (console)

1. Go to **aws.amazon.com** → **Create an AWS Account**.
2. Enter an email address and an account name (e.g. `vs-lab-yourname`). Enter the code sent to your email.

3. Set the **root user** password. This login can do everything — keep it safe.
4. Fill in contact details, then add a payment card (AWS makes a small verification charge that is refunded).
5. Verify your phone number with the SMS or call code.
6. Choose an account plan. **Free plan** is fine for this lab: you get sign-up credits and no charges until you upgrade. If a service says it is not available on the Free plan, you can upgrade later.
7. Choose **Basic support (free)** and finish. Sign in to the **AWS Management Console**.

Make the account safe (do this once)

1. Top right, click your account name → **Security credentials** → **Assign MFA device**. Use an authenticator app on your phone.
2. Search **Budgets** → **Create budget** → template **Zero spend budget** → enter your email → **Create**. You get an email the moment anything costs money.
3. Search **IAM** → **Users** → **Create user** → name `lab-admin` → tick **Provide user access to the AWS Management Console** → next → **Attach policies directly** → tick `AdministratorAccess` → **Create user**. Use this user from now on, not root.
4. Open `lab-admin` → **Security credentials** → **Create access key** → choose **Command Line Interface (CLI)** → copy the **Access key ID** and **Secret access key**. You need them in the next section.
5. Top right Region menu → choose **Asia Pacific (Mumbai) ap-south-1**. Always check this before creating anything.

> In a company you will not create users like this. You sign in through **IAM Identity Center** (single sign-on) and use `aws configure sso`. Access keys are fine for a personal lab account — never share or commit them to Git.

27 Before you start: set up the command line

All commands are run in the **PyCharm Terminal** (bottom bar → Terminal). On Mac it opens zsh; on Windows it opens PowerShell. No PyCharm? Use Terminal on Mac or PowerShell on Windows — same commands.

Mac — install and configure

```
cd ~
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
aws --version

aws configure
#   AWS Access Key ID:     <paste access key id>
#   AWS Secret Access Key: <paste secret key>
#   Default region name:   ap-south-1
#   Default output format: json

aws sts get-caller-identity          # shows your account and lab-admin user

mkdir -p ~/aws-network-lab
cd ~/aws-network-lab
```

Windows — install and configure

```
cd ~
msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
# finish the installer, then CLOSE and REOPEN the PyCharm Terminal
aws --version

aws configure
#   AWS Access Key ID:     <paste access key id>
#   AWS Secret Access Key: <paste secret key>
#   Default region name:   ap-south-1
#   Default output format: json

aws sts get-caller-identity          # shows your account and lab-admin user

mkdir ~\aws-network-lab
cd ~\aws-network-lab
```

Keep the same terminal window open for all the build steps. Each step saves IDs into variables (`$VPC` , `$IGW` ...) that later steps use. If you close the terminal, the variables are lost — copy the IDs from the console and set them again.

No install possible? Open **CloudShell** (the `>_` icon in the console top bar). It already has the CLI and your login. Use the **Mac** commands there.

28 Build 1 · Create the VPC

Console

1. Search **VPC** → left menu **Your VPCs** → **Create VPC**.
2. **Resources to create:** VPC only. **Name tag:** `hr-lab-vpc` .
3. **IPv4 CIDR block:** IPv4 CIDR manual input → `10.2.0.0/16` . **IPv6:** No IPv6 CIDR block. → **Create VPC**.
4. Select the VPC → **Actions** → **Edit VPC settings** → tick **Enable DNS hostnames** → **Save**.

Command line — Mac

```
VPC=$(aws ec2 create-vpc --cidr-block 10.2.0.0/16 --tag-specifications 'ResourceType=vpc,Tags=
[{Key=Name,Value=hr-lab-vpc}]' --query Vpc.VpcId --output text)
aws ec2 modify-vpc-attribute --vpc-id $VPC --enable-dns-hostnames Value=true
echo $VPC
```

Command line — Windows (PowerShell)

```
$VPC = aws ec2 create-vpc --cidr-block 10.2.0.0/16 --tag-specifications 'ResourceType=vpc,Tags=
[{Key=Name,Value=hr-lab-vpc}]' --query Vpc.VpcId --output text
aws ec2 modify-vpc-attribute --vpc-id $VPC --enable-dns-hostnames Value=true
echo $VPC
```

Shortcut you will see in the console: **VPC and more** builds the VPC, subnets, gateways and route tables in one screen. Use it later — first build each piece by hand to see what it does.

29 Build 2 · Create four subnets

Console

1. Left menu **Subnets** → **Create subnet** → **VPC ID:** `hr-lab-vpc`.
2. Subnet 1: name `public-a`, AZ `ap-south-1a`, CIDR `10.2.0.0/24`. Click **Add new subnet** for each of the next three.
3. Subnet 2: `public-b`, `ap-south-1b`, `10.2.1.0/24`. Subnet 3: `app-a`, `ap-south-1a`, `10.2.16.0/20`. Subnet 4: `app-b`, `ap-south-1b`, `10.2.32.0/20`. → **Create subnet**.
4. Select `public-a` → **Actions** → **Edit subnet settings** → tick **Enable auto-assign public IPv4 address** → **Save**. Repeat for `public-b`.

Command line — Mac

```
PUB_A=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.2.0.0/24 --availability-zone ap-south-
1a --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=public-a}]' --query
Subnet.SubnetId --output text)
PUB_B=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.2.1.0/24 --availability-zone ap-south-
1b --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=public-b}]' --query
Subnet.SubnetId --output text)
APP_A=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.2.16.0/20 --availability-zone ap-south-
1a --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=app-a}]' --query Subnet.SubnetId
--output text)
APP_B=$(aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.2.32.0/20 --availability-zone ap-south-
1b --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=app-b}]' --query Subnet.SubnetId
--output text)

# public subnets give new servers a public IP automatically
aws ec2 modify-subnet-attribute --subnet-id $PUB_A --map-public-ip-on-launch
aws ec2 modify-subnet-attribute --subnet-id $PUB_B --map-public-ip-on-launch

# see the free IPs in each subnet: 251 and 4091
aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC" --query 'Subnets[].
[Tags[0].Value,CidrBlock,AvailableIpAddressCount]' --output table
```

Command line — Windows (PowerShell)

```
$PUB_A = aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.2.0.0/24 --availability-zone ap-south-
1a --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=public-a}]' --query
Subnet.SubnetId --output text
$PUB_B = aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.2.1.0/24 --availability-zone ap-south-
1b --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=public-b}]' --query
Subnet.SubnetId --output text
$APP_A = aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.2.16.0/20 --availability-zone ap-
south-1a --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=app-a}]' --query
Subnet.SubnetId --output text
$APP_B = aws ec2 create-subnet --vpc-id $VPC --cidr-block 10.2.32.0/20 --availability-zone ap-
south-1b --tag-specifications 'ResourceType=subnet,Tags=[{Key=Name,Value=app-b}]' --query
Subnet.SubnetId --output text

# public subnets give new servers a public IP automatically
aws ec2 modify-subnet-attribute --subnet-id $PUB_A --map-public-ip-on-launch
aws ec2 modify-subnet-attribute --subnet-id $PUB_B --map-public-ip-on-launch

# see the free IPs in each subnet: 251 and 4091
aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC" --query 'Subnets[].
[Tags[0].Value,CidrBlock,AvailableIpAddressCount]' --output table
```

> At this point the "public" subnets are not public yet. They have no route to the internet. Steps 3 and 4 make them public.

30 Build 3 · Create and attach the Internet Gateway

Console

1. Left menu **Internet gateways** → **Create internet gateway** → name `hr-lab-igw` → **Create**.
2. On the green banner (or **Actions**) → **Attach to VPC** → choose `hr-lab-vpc` → **Attach**.
3. The state changes from Detached to **Attached**.

Command line — Mac

```
IGW=$(aws ec2 create-internet-gateway --tag-specifications 'ResourceType=internet-gateway,Tags=
[{Key=Name,Value=hr-lab-igw}]' --query InternetGateway.InternetGatewayId --output text)
aws ec2 attach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC
echo $IGW
```

Command line — Windows (PowerShell)

```
$IGW = aws ec2 create-internet-gateway --tag-specifications 'ResourceType=internet-gateway,Tags=
[{Key=Name,Value=hr-lab-igw}]' --query InternetGateway.InternetGatewayId --output text
aws ec2 attach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC
echo $IGW
```

31 Build 4 · Public route table: 0.0.0.0/0 → IGW

Console

1. Left menu **Route tables** → **Create route table** → name `public-rt`, VPC `hr-lab-vpc` → **Create**.

2. **Routes** tab → **Edit routes** → **Add route** → Destination `0.0.0.0/0`, Target **Internet Gateway** → `hr-lab-igw` → **Save changes**.
3. **Subnet associations** tab → **Edit subnet associations** → tick `public-a` and `public-b` → **Save associations**.

Command line — Mac

```
PUB_RT=$(aws ec2 create-route-table --vpc-id $VPC --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=public-rt}]' --query RouteTable.RouteTableId --output text)
aws ec2 create-route --route-table-id $PUB_RT --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
aws ec2 associate-route-table --route-table-id $PUB_RT --subnet-id $PUB_A
aws ec2 associate-route-table --route-table-id $PUB_RT --subnet-id $PUB_B
```

Command line — Windows (PowerShell)

```
$PUB_RT = aws ec2 create-route-table --vpc-id $VPC --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=public-rt}]' --query RouteTable.RouteTableId --output text
aws ec2 create-route --route-table-id $PUB_RT --destination-cidr-block 0.0.0.0/0 --gateway-id $IGW
aws ec2 associate-route-table --route-table-id $PUB_RT --subnet-id $PUB_A
aws ec2 associate-route-table --route-table-id $PUB_RT --subnet-id $PUB_B
```

> This is the moment `public-a` and `public-b` become **public subnets** — only because of this one route.

32 Build 5 · NAT Gateway and private route table

Console

1. Left menu **NAT gateways** → **Create NAT gateway** → name `hr-lab-nat`.
2. **Availability mode:** Zonal. **Subnet:** `public-a`. **Connectivity type:** Public. Click **Allocate Elastic IP** → **Create NAT gateway**.
3. Wait until **State = Available** (1–3 minutes).
4. **Route tables** → **Create route table** → name `app-rt`, VPC `hr-lab-vpc` → **Create**.
5. **Routes** → **Edit routes** → **Add route** → `0.0.0.0/0` → Target **NAT Gateway** → `hr-lab-nat` → **Save**.
6. **Subnet associations** → tick `app-a` and `app-b` → **Save associations**.

Command line — Mac

```
EIP=$(aws ec2 allocate-address --domain vpc --query AllocationId --output text)
NAT=$(aws ec2 create-nat-gateway --subnet-id $PUB_A --allocation-id $EIP --tag-specifications 'ResourceType=natgateway,Tags=[{Key=Name,Value=hr-lab-nat}]' --query NatGateway.NatGatewayId --output text)
aws ec2 wait nat-gateway-available --nat-gateway-ids $NAT

APP_RT=$(aws ec2 create-route-table --vpc-id $VPC --tag-specifications 'ResourceType=route-table,Tags=[{Key=Name,Value=app-rt}]' --query RouteTable.RouteTableId --output text)
aws ec2 create-route --route-table-id $APP_RT --destination-cidr-block 0.0.0.0/0 --nat-gateway-id $NAT
aws ec2 associate-route-table --route-table-id $APP_RT --subnet-id $APP_A
aws ec2 associate-route-table --route-table-id $APP_RT --subnet-id $APP_B
```

Command line — Windows (PowerShell)

```
$EIP = aws ec2 allocate-address --domain vpc --query AllocationId --output text
$NAT = aws ec2 create-nat-gateway --subnet-id $PUB_A --allocation-id $EIP --tag-specifications
'ResourceType=natgateway,Tags=[{Key=Name,Value=hr-lab-nat}]' --query NatGateway.NatGatewayId --
output text
aws ec2 wait nat-gateway-available --nat-gateway-ids $NAT

$APP_RT = aws ec2 create-route-table --vpc-id $VPC --tag-specifications 'ResourceType=route-
table,Tags=[{Key=Name,Value=app-rt}]' --query RouteTable.RouteTableId --output text
aws ec2 create-route --route-table-id $APP_RT --destination-cidr-block 0.0.0.0/0 --nat-gateway-id
$NAT
aws ec2 associate-route-table --route-table-id $APP_RT --subnet-id $APP_A
aws ec2 associate-route-table --route-table-id $APP_RT --subnet-id $APP_B
```

We use the **zonal** NAT here because every step is easy to see (it sits in `public-a` with its own Elastic IP). To try the **regional** one instead: `aws ec2 create-nat-gateway --vpc-id $VPC --availability-mode regional`, and no public subnet or Elastic IP is needed.

33 Build 6 · S3 gateway endpoint

Console

1. Left menu **Endpoints** → **Create endpoint** → name `s3-endpoint`, type **AWS services**.
2. Search `s3` → pick `com.amazonaws.ap-south-1.s3` with **Type = Gateway**.
3. **VPC:** `hr-lab-vpc`. **Route tables:** tick `app-rt`. Policy: **Full access** → **Create endpoint**.
4. Open `app-rt` → **Routes**: a new line `pl-xxxx (com.amazonaws.ap-south-1.s3) → vpce-xxxx` has appeared.

Command line — Mac

```
VPCE=$(aws ec2 create-vpc-endpoint --vpc-id $VPC --service-name com.amazonaws.ap-south-1.s3 --
route-table-ids $APP_RT --query VpcEndpoint.VpcEndpointId --output text)
aws ec2 describe-route-tables --route-table-ids $APP_RT --query 'RouteTables[0].Routes' --output
table
```

Command line — Windows (PowerShell)

```
$VPCE = aws ec2 create-vpc-endpoint --vpc-id $VPC --service-name com.amazonaws.ap-south-1.s3 --
route-table-ids $APP_RT --query VpcEndpoint.VpcEndpointId --output text
aws ec2 describe-route-tables --route-table-ids $APP_RT --query 'RouteTables[0].Routes' --output
table
```

34 Build 7 · Security groups

Console

1. Left menu **Security groups** → **Create security group** → name `web-sg`, description `HTTP from internet`, VPC `hr-lab-vpc`.
2. **Inbound rules** → **Add rule** → Type **HTTP** (port 80), Source **Anywhere-IPv4** → **Create security group**.

3. Create a second one: name `app-sg`, description `No inbound`, VPC `hr-lab-vpc`, **no inbound rules** → **Create**.

Command line — Mac

```
WEB_SG=$(aws ec2 create-security-group --group-name web-sg --description "HTTP from internet" --
vpc-id $VPC --query GroupId --output text)
aws ec2 authorize-security-group-ingress --group-id $WEB_SG --protocol tcp --port 80 --cidr
0.0.0.0/0

APP_SG=$(aws ec2 create-security-group --group-name app-sg --description "No inbound" --vpc-id $VPC
--query GroupId --output text)
```

Command line — Windows (PowerShell)

```
$WEB_SG = aws ec2 create-security-group --group-name web-sg --description "HTTP from internet" --
vpc-id $VPC --query GroupId --output text
aws ec2 authorize-security-group-ingress --group-id $WEB_SG --protocol tcp --port 80 --cidr
0.0.0.0/0

$APP_SG = aws ec2 create-security-group --group-name app-sg --description "No inbound" --vpc-id
$VPC --query GroupId --output text
```

> `app-sg` has no inbound rules, so **nothing** can start a connection to the private server. Outbound is open by default, so it can still go out through the NAT.

35 Build 8 · Launch two servers

Console

1. Search **EC2** → **Launch instances**. Name `web-public`. AMI **Amazon Linux 2023**. Instance type `t3.micro`.
2. **Key pair:** Proceed without a key pair (we will not log in).
3. **Network settings** → **Edit** → VPC `hr-lab-vpc`, Subnet `public-a`, **Auto-assign public IP: Enable**, **Select existing security group** → `web-sg`.
4. **Advanced details** → **User data** → paste the `web.sh` script shown below → **Launch instance**.
5. Launch a second one: name `app-private`, Subnet `app-a`, **Auto-assign public IP: Disable**, security group `app-sg`, user data = the `nat.sh` script → **Launch instance**.

Command line — Mac

```
cat > web.sh <<'EOF'
#!/bin/bash
mkdir -p /var/www
echo "Hello from the public subnet. My private IP is $(hostname -I)" > /var/www/index.html
nohup python3 -m http.server 80 --directory /var/www > /dev/null 2>&1 &
EOF

cat > nat.sh <<'EOF'
#!/bin/bash
sleep 30
CODE=$(curl -s -o /dev/null -w "%{http_code}" https://aws.amazon.com)
echo "NAT TEST: internet answered with HTTP $CODE" | tee /dev/console
EOF

WEB=$(aws ec2 run-instances --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-
kernel-default-x86_64 --instance-type t3.micro --subnet-id $PUB_A --security-group-ids $WEB_SG --
user-data file://web.sh --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-
public}]' --query 'Instances[0].InstanceId' --output text)
APP=$(aws ec2 run-instances --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-ami-
kernel-default-x86_64 --instance-type t3.micro --subnet-id $APP_A --security-group-ids $APP_SG --
user-data file://nat.sh --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=app-
private}]' --query 'Instances[0].InstanceId' --output text)
aws ec2 wait instance-running --instance-ids $WEB $APP
```

Command line — Windows (PowerShell)

```
$web = @'
#!/bin/bash
mkdir -p /var/www
echo "Hello from the public subnet. My private IP is $(hostname -I)" > /var/www/index.html
nohup python3 -m http.server 80 --directory /var/www > /dev/null 2>&1 &
'@
[IO.File]::WriteAllText("$PWD\web.sh", ($web -replace "`r",""))

$nat = @'
#!/bin/bash
sleep 30
CODE=$(curl -s -o /dev/null -w "%{http_code}" https://aws.amazon.com)
echo "NAT TEST: internet answered with HTTP $CODE" | tee /dev/console
'@
[IO.File]::WriteAllText("$PWD\nat.sh", ($nat -replace "`r",""))

$WEB = aws ec2 run-instances --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-
ami-kernel-default-x86_64 --instance-type t3.micro --subnet-id $PUB_A --security-group-ids $WEB_SG
--user-data file://web.sh --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=web-
public}]' --query 'Instances[0].InstanceId' --output text
$APP = aws ec2 run-instances --image-id resolve:ssm:/aws/service/ami-amazon-linux-latest/al2023-
ami-kernel-default-x86_64 --instance-type t3.micro --subnet-id $APP_A --security-group-ids $APP_SG
--user-data file://nat.sh --tag-specifications 'ResourceType=instance,Tags=[{Key=Name,Value=app-
private}]' --query 'Instances[0].InstanceId' --output text
aws ec2 wait instance-running --instance-ids $WEB $APP
```

User data is a script the server runs once when it first starts. `web.sh` starts a tiny web page. `nat.sh` tries to reach the internet and writes the result to the server's system log. On Windows, the script is saved with Linux line endings — otherwise Linux cannot run it.

36 Build 9 · Check that it works

Check	Console	Expected
IGW works (in and out)	EC2 → `web-public` → copy **Public IPv4 address** → open `http://<that IP>` in a browser (http, not https)	"Hello from the public subnet. My private IP is 10.2.0.x"
Private server has no public IP	EC2 → `app-private` → details	Only a private IP `10.2.16.x`; Public IPv4 is empty
NAT works (out only)	`app-private` → **Actions** → **Monitor and troubleshoot** → **Get system log** (wait ~5 min after launch)	A line `NAT TEST: internet answered with HTTP 200`
Devices use IPs	VPC → **Subnets** → column **Available IPv4 addresses**	`app-a`: 4090 (1 used). `public-a`: 249 (server + NAT)

Command line — Mac

```
aws ec2 describe-instances --instance-ids $WEB $APP --query 'Reservations[].Instances[].
[Tags[0].Value,PrivateIpAddress,PublicIpAddress]' --output table

# wait ~5 minutes after launch, then:
aws ec2 get-console-output --instance-id $APP --latest --output text | grep "NAT TEST"

aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC" --query 'Subnets[].
[Tags[0].Value,AvailableIpAddressCount]' --output table
```

Command line — Windows (PowerShell)

```
aws ec2 describe-instances --instance-ids $WEB $APP --query 'Reservations[].Instances[].
[Tags[0].Value,PrivateIpAddress,PublicIpAddress]' --output table

# wait ~5 minutes after launch, then:
aws ec2 get-console-output --instance-id $APP --latest --output text | Select-String "NAT TEST"

aws ec2 describe-subnets --filters "Name=vpc-id,Values=$VPC" --query 'Subnets[].
[Tags[0].Value,AvailableIpAddressCount]' --output table
```

Try breaking it to learn: delete the `0.0.0.0/0 → NAT` route from `app-rt`, then launch one more server like `app-private`. Its log shows `HTTP 000` — the request never got out. Put the route back afterwards, and terminate the extra server.

37 Build 10 · Clean up (same day)

Console

1. EC2 → select `web-public` and `app-private` → **Instance state** → **Terminate**. Wait until it shows Terminated.
2. VPC → **NAT gateways** → `hr-lab-nat` → **Actions** → **Delete**. Wait until it shows Deleted (a few minutes).

3. VPC → **Elastic IPs** → select the address → **Actions** → **Release Elastic IP addresses**.
4. VPC → **Your VPCs** → `hr-lab-vpc` → **Actions** → **Delete VPC** → type delete to confirm. This also removes the subnets, route tables, Internet Gateway, security groups and the S3 endpoint.
5. Billing → **Bills** next day: check nothing is still running.

Command line — Mac

```
aws ec2 terminate-instances --instance-ids $WEB $APP
aws ec2 wait instance-terminated --instance-ids $WEB $APP

aws ec2 delete-vpc-endpoints --vpc-endpoint-ids $VPCE
aws ec2 delete-nat-gateway --nat-gateway-id $NAT
aws ec2 wait nat-gateway-deleted --nat-gateway-ids $NAT
aws ec2 release-address --allocation-id $EIP

aws ec2 detach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC
aws ec2 delete-internet-gateway --internet-gateway-id $IGW

aws ec2 delete-subnet --subnet-id $PUB_A
aws ec2 delete-subnet --subnet-id $PUB_B
aws ec2 delete-subnet --subnet-id $APP_A
aws ec2 delete-subnet --subnet-id $APP_B
aws ec2 delete-route-table --route-table-id $PUB_RT
aws ec2 delete-route-table --route-table-id $APP_RT
aws ec2 delete-security-group --group-id $WEB_SG
aws ec2 delete-security-group --group-id $APP_SG
aws ec2 delete-vpc --vpc-id $VPC

cd ~
```

Command line — Windows (PowerShell)

```
aws ec2 terminate-instances --instance-ids $WEB $APP
aws ec2 wait instance-terminated --instance-ids $WEB $APP

aws ec2 delete-vpc-endpoints --vpc-endpoint-ids $VPCE
aws ec2 delete-nat-gateway --nat-gateway-id $NAT
aws ec2 wait nat-gateway-deleted --nat-gateway-ids $NAT
aws ec2 release-address --allocation-id $EIP

aws ec2 detach-internet-gateway --internet-gateway-id $IGW --vpc-id $VPC
aws ec2 delete-internet-gateway --internet-gateway-id $IGW

aws ec2 delete-subnet --subnet-id $PUB_A
aws ec2 delete-subnet --subnet-id $PUB_B
aws ec2 delete-subnet --subnet-id $APP_A
aws ec2 delete-subnet --subnet-id $APP_B
aws ec2 delete-route-table --route-table-id $PUB_RT
aws ec2 delete-route-table --route-table-id $APP_RT
aws ec2 delete-security-group --group-id $WEB_SG
aws ec2 delete-security-group --group-id $APP_SG
aws ec2 delete-vpc --vpc-id $VPC

cd ~
```

The order matters: servers first, then NAT and Elastic IP, then the gateway, then subnets and route tables, and the VPC last. AWS refuses to delete something that is still in use — the error message tells you what is still attached.

38 Key takeaways

1. An IP address identifies a device on a network. Every device (every network interface) needs its own.
2. More devices = more IPs. Containers, pods, load balancers and endpoints all take IPs in AWS.
3. A CIDR range is a block of IPs: /16 = 65,536, /20 = 4,096, /24 = 256. AWS keeps 5 per subnet.
4. Size subnets for peak devices × 2–4. When unsure, go bigger.
5. Region → AZ → data centre. Use at least 2 AZs. One AWS account per team per environment.
6. VPC = your private network. Plan company ranges once so they never overlap.
7. Only the load balancer is public. Private apps reach AWS services through endpoints and the internet through NAT.
8. Security groups open ports; route tables decide paths. Timeouts usually mean one of the two is missing.